

Chess Engine – Software Engineering Project Documentation

By: Dylan Van

Table of Contents

Introduction.....	3
Requirements Analysis.....	5
Functional Requirements.....	5
Non-functional Requirements.....	6
Constraints.....	7
Research.....	9
Project Management.....	14
Gantt Chart.....	14
Risk Analysis.....	15
System Design.....	17
Use Case Diagram.....	17
UML Class Diagram.....	18
Data Flow Diagram.....	19
System Architecture Design.....	20
Menu Design Wireframe.....	21
Interface Design.....	22
Flowchart.....	23
Board Structure Diagram.....	24
Algorithms.....	25
Development Log.....	29
Testing.....	34
Implementation.....	12
Evaluation.....	40
References.....	46
Appendix.....	50

1. Introduction

1.1 Purpose

The purpose of this chess game is to make it playable for beginners and help them get better at it by using the difficulties such as easy mode and medium. This works as they play against an AI as an opponent. I am developing chess as it is my favourite board game since I was younger and want to make more people play this game as it requires a lot of thinking but also it is fun and competitive.

1.2 Problem Statement

Many beginner and intermediate chess players struggle to analyse their gameplay and identify mistakes. Existing tools are often complex or not easily accessible, limiting user improvement.

1.3 Proposed Solution

This project aims to develop a web-based application called ChessTrainer, which allows users to play chess against an AI engine and receive feedback on their moves. The system stores game data and provides analysis to support user improvement.

1.4 Target Users

The primary target users of this project are beginner chess players who are learning the fundamentals of the game and wish to develop their understanding of chess rules and basic strategies. The application is also designed for intermediate players seeking to improve their decision-making skills, tactical awareness and overall gameplay through practice against an AI opponent. In addition, students interested in learning chess strategies can use the program as an educational tool to analyse board positions, understand move evaluation scores and develop critical thinking and problem-solving skills in an engaging environment.

1.5 Success Criteria

The project will be considered successful if it allows users to play a complete game of chess from start to finish with all standard chess rules functioning correctly. The system should include an AI opponent that automatically responds to the player's moves, creating an interactive and challenging gameplay experience. An evaluation score should be displayed during the game to provide players with insight into the current board position and which side holds the advantage. Additionally, the program should offer multiple difficulty levels, allowing users of different skill levels to adjust the challenge and improve their gameplay over time. These success criteria will ensure the application is functional, engaging and meets the objectives of the project.

2. Requirements Analysis

Requirements analysis identifies what the software must do and the standards it must meet. This process ensures the Chess AI Trainer satisfies the needs of users while remaining practical to develop using the available tools and resources.

2.1 Functional Requirements

Functional requirements describe the specific features and operations the system must perform.

FR1 – Move Pieces

The system must allow users to move chess pieces around the board according to the rules of chess. Players select a piece and choose a destination square. Once a valid move is chosen, the board updates to reflect the new position.

This requirement is achieved through the interaction between the Board, Move, and Piece classes, which manage piece positions and movement across the chessboard.

FR2 – Validate Legal Moves

The system must ensure that players can only perform legal chess moves. Invalid moves should be rejected automatically.

For example, a bishop may only move diagonally, while a knight moves in an L-shape. The move validation system checks whether a selected move follows the rules of the piece before allowing it to be executed.

This functionality is implemented in the `calc_moves()` and `valid_move()` methods within the Board class.

FR3 – AI Opponent

The system must provide an artificial intelligence opponent capable of automatically generating moves after the player completes their turn.

The AI analyses the current board position and selects a move based on a board evaluation algorithm. This allows users to practise chess without requiring a second player.

The feature is implemented through the ChessAI class.

FR4 – Difficulty Selection

The system must allow users to select different difficulty levels before beginning a game.

Difficulty settings provide users with varying levels of challenge and improve the educational value of the application. Beginner players can select Easy mode, while more experienced users can select Medium mode.

Difficulty selection is available through the start menu and affects the AI decision-making process.

FR5 – Game Reset

The system must provide a reset function that returns the board to its starting position.

This allows users to quickly begin a new game without restarting the application. The reset feature improves usability and assists during testing and debugging.

The feature is activated using the keyboard shortcut and reinitialises all game objects.

FR6 – Piece Capture

The system must support the capture of opponent pieces.

When a piece moves onto a square occupied by an opposing piece, the opposing piece is removed from the board. Capture functionality is a fundamental requirement of chess and contributes to gameplay realism. The system also plays a capture sound effect to provide visual and audio feedback to the user.

2.2 Non-Functionable Requirements

Non-functional requirements describe the quality standards the software must achieve.

NFR1 – Fast Performance

The application must respond quickly to user actions and AI calculations. Moves should occur immediately after selection, and the AI should generate responses within a reasonable time frame. Slow performance would negatively affect the user experience.

Python and Pygame provide sufficient performance for a school-scale chess application.

NFR2 – Easy to Use

The interface must be simple and intuitive for beginner users.

Players should be able to start a game, move pieces, select a difficulty level and reset the board without requiring technical knowledge. Clear graphics and straightforward controls improve accessibility.

NFR3 – Reliable

The system must operate consistently without crashes or unexpected behaviour.

Reliability is particularly important during gameplay because incorrect move validation or application crashes would reduce user confidence and affect the educational value of the software.

Extensive testing was conducted to ensure reliability.

NFR4 – Maintainable

The software must be structured in a way that allows future modifications and improvements.

Object-oriented programming principles were used to separate responsibilities into classes such as Board, Game, Piece, and ChessAI. This modular design makes the program easier to debug, extend and maintain.

2.3 Constraints

Constraints are limitations that affect the development of the project.

C1 – Python

The project was required to be developed using Python due to its simplicity, readability and strong support for object-oriented programming. Python also provides numerous libraries suitable for game development and artificial intelligence.

C2 – Pygame

The graphical user interface was developed using the Pygame library. While Pygame provides effective tools for creating games, it is less advanced than professional game engines such as Unity or Unreal Engine. This limited some graphical features that could be implemented.

C3 – School Hardware

The software needed to run efficiently on standard school computers without requiring specialised hardware. As a result, the AI algorithm was intentionally kept lightweight to ensure smooth performance across all available devices.

3. Research

Before developing the Chess AI Trainer, research was conducted into chess rules, artificial intelligence algorithms and existing chess platforms. This research helped determine the features, design decisions and implementation methods used throughout the project.

3.1 Chess Rules Research

A thorough understanding of chess rules was required before development could begin. The application needed to accurately replicate the movement patterns and gameplay mechanics of a real chess match.

Research was conducted into the movement rules of all six chess pieces:

- Pawn
- Rook
- Knight
- Bishop
- Queen
- King

Special chess rules were also investigated, including:

- Castling
- Pawn promotion
- En passant
- Check
- Checkmate
- Stalemate

Understanding these rules was essential because the move validation system relies on them to determine whether a player's move is legal. Incorrect implementation would result in unrealistic gameplay and reduce the educational value of the application.

The research also highlighted the importance of preventing illegal moves that place a player's king in check. This influenced the design of the move generation and validation algorithms within the Board class.

3.2 AI Algorithms Research

One of the main goals of the project was to provide a computer-controlled opponent. Several artificial intelligence approaches were investigated before selecting the final implementation.

1. Minimax Algorithm

The Minimax algorithm is one of the most widely used decision-making algorithms in chess engines.

Minimax works by exploring future board positions and assuming that both players make the best possible moves. The algorithm attempts to maximise the AI's advantage while minimising the opponent's advantage.

Advantages:

- Produces stronger gameplay
- Can analyse multiple future moves
- Used in professional chess engines

Disadvantages:

- Computationally expensive
- Difficult to implement efficiently
- Requires significant processing power

Due to the project's time constraints and the limitations of school hardware, a full Minimax implementation was considered too complex for the current version of the application.

However, research into Minimax influenced the design of the evaluation system and may be implemented in future versions.

2. Random Move AI

Random Move AI selects a legal move at random from all available moves.

Advantages:

- Very easy to implement
- Fast performance
- Suitable for beginners

Disadvantages:

- Makes poor strategic decisions
- Easily defeated
- Provides limited challenge

While Random Move AI offers excellent performance, it does not provide a realistic chess experience. Therefore, it was not selected as the primary AI strategy.

3. Evaluation Functions

An evaluation function assigns a numerical value to a board position.

The AI analyses the pieces remaining on the board and calculates which player has an advantage.

Research showed that most chess engines use piece values similar to:

Piece	Value
Pawn	1
Knight	3
Bishop	3
Rook	5
Queen	9
King	Very High Value

This approach was adopted in the project's ChessAI class.

The evaluation score is displayed on-screen to provide users with feedback regarding which side currently holds the advantage.

The evaluation function provided a balance between simplicity and effectiveness while remaining suitable for school-level hardware.

3.3 Similar Products

Research was conducted into existing chess platforms to identify useful features and industry standards.

Chess.com

Chess.com is one of the world's largest online chess platforms.

Key features include:

- Online multiplayer
- Computer opponents
- Puzzle training
- Performance analysis
- Player rankings

The platform demonstrated the importance of providing AI opponents for practice and skill development.

However, many of its advanced features were beyond the scope of this project.

Lichess

Lichess is a free and open-source online chess platform.

Key features include:

- Free analysis tools
- Online play
- Opening explorer
- Training exercises
- Study resources

Research into Lichess highlighted the value of simplicity and accessibility. Its clean user interface influenced the design goal of creating an easy-to-use application for beginner players.

Stockfish

Stockfish is one of the strongest chess engines in the world.

Key features include:

- Advanced search algorithms
- Deep position analysis
- Extremely high playing strength
- Open-source development

Although Stockfish is significantly more sophisticated than this project, researching its architecture provided insight into how chess engines evaluate positions and generate moves.

The project uses a simplified evaluation-based AI inspired by the principles used in professional engines such as Stockfish.

Research Conclusion

The research phase demonstrated that successful chess applications require accurate rule implementation, efficient move validation and intelligent decision-making. While advanced algorithms such as Minimax and Stockfish-style search were beyond the scope of this project, the evaluation-based AI provides a suitable balance between performance and functionality. Research into existing chess platforms also influenced the decision to prioritise usability, educational value and accessibility for beginner players.

4. Project Management

4.1 Gantt Chart



4.2 Risk Analysis

Risk	Likelihood	Impact	Mitigation Strategy
AI does not function correctly	Medium	High	Thorough testing of move generation and evaluation functions throughout development.
Chess rules implemented incorrectly	Medium	High	Research official chess rules and perform extensive gameplay testing.
Program crashes due to coding errors	Medium	High	Use debugging tools, code reviews and regular testing after each feature is added.
Project falls behind schedule	Medium	High	Follow project timeline and complete milestones progressively.
Loss of source code	Low	High	Use Git and GitHub for version control and regular backups.
Hardware failure	Low	High	Store project files in cloud storage and maintain backup copies.
Pygame compatibility issues	Low	Medium	Test the application on multiple school computers during development.
User interface becomes difficult to use	Medium	Medium	Gather feedback from users and simplify controls where necessary.
AI becomes too easy or too difficult	Medium	Medium	Implement difficulty settings and test with different players.
Checkmate and move validation errors	Medium	High	Create test cases for special scenarios such as checkmate, stalemate and castling.
Performance issues when calculating moves	Low	Medium	Optimise move generation algorithms and avoid unnecessary calculations.
Insufficient documentation	Medium	Medium	Update documentation regularly throughout the project lifecycle.

Risk Matrix

High Impact / High Priority

- AI malfunction
- Chess rule implementation errors
- Program crashes
- Project delays
- Source code loss

Medium Impact / Medium Priority

- User interface issues
- Difficulty balancing
- Documentation issues

Low Priority

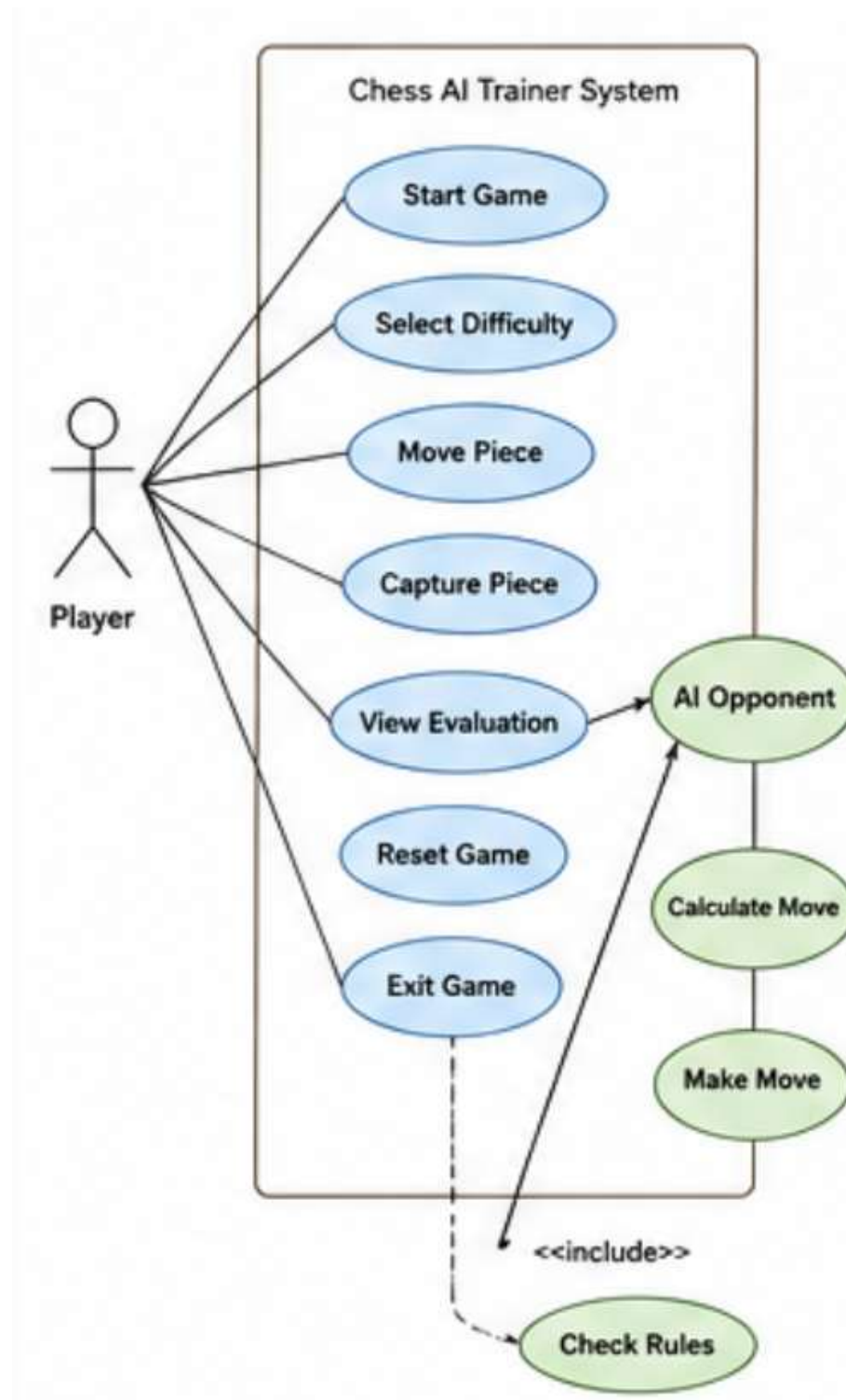
- Hardware failures
- Pygame compatibility problems
- Minor performance issues

Risk Evaluation

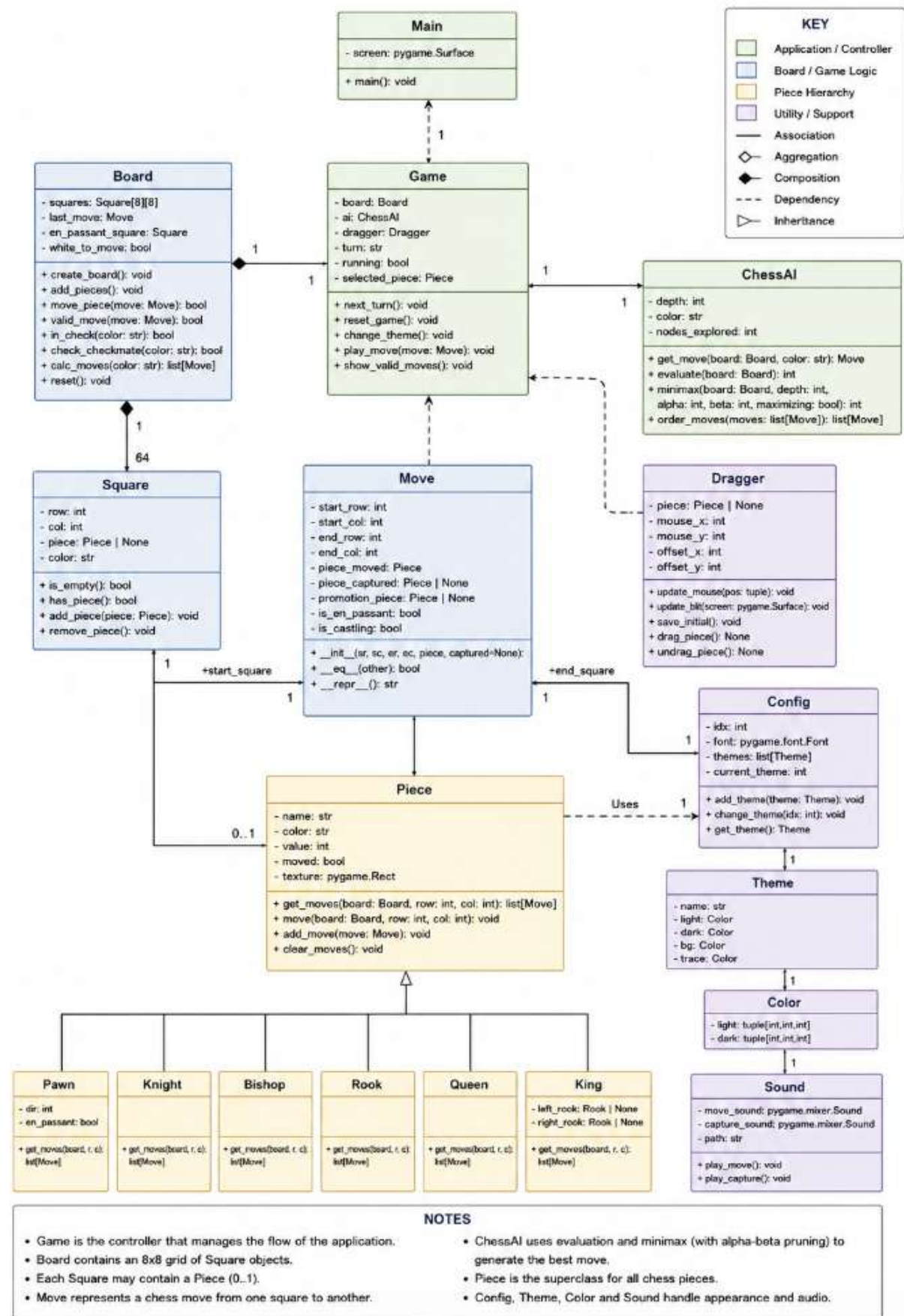
The greatest risks to the success of the project are software defects and incorrect implementation of chess mechanics. Since the application relies heavily on move validation and AI decision-making, errors in these systems could significantly impact functionality and user experience. To minimise these risks, iterative testing, version control using GitHub and regular progress reviews will be conducted throughout development. Maintaining backups and following a structured development schedule will further reduce the likelihood of project failure.

5. System Design

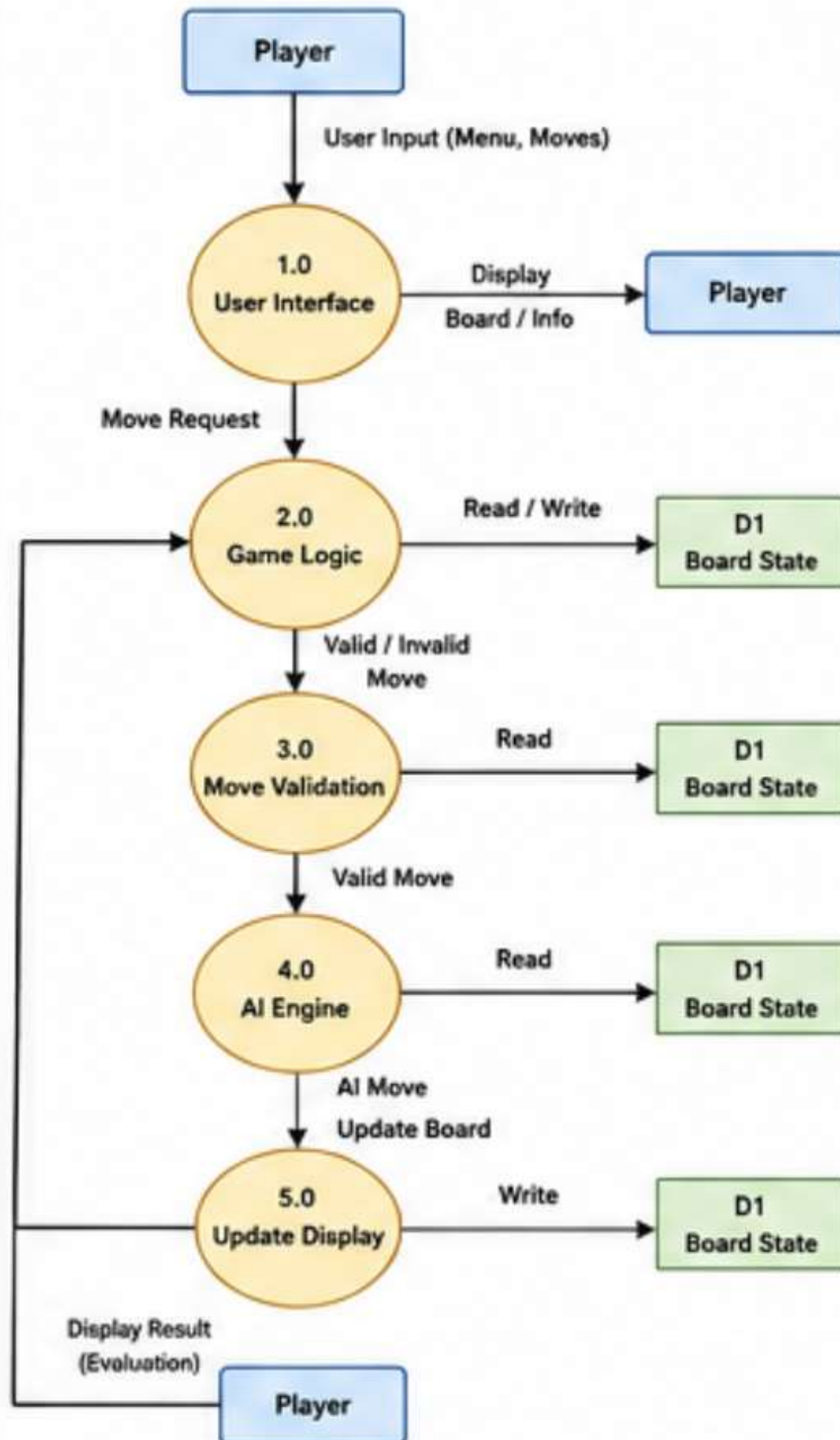
5.1 Use Case Diagram



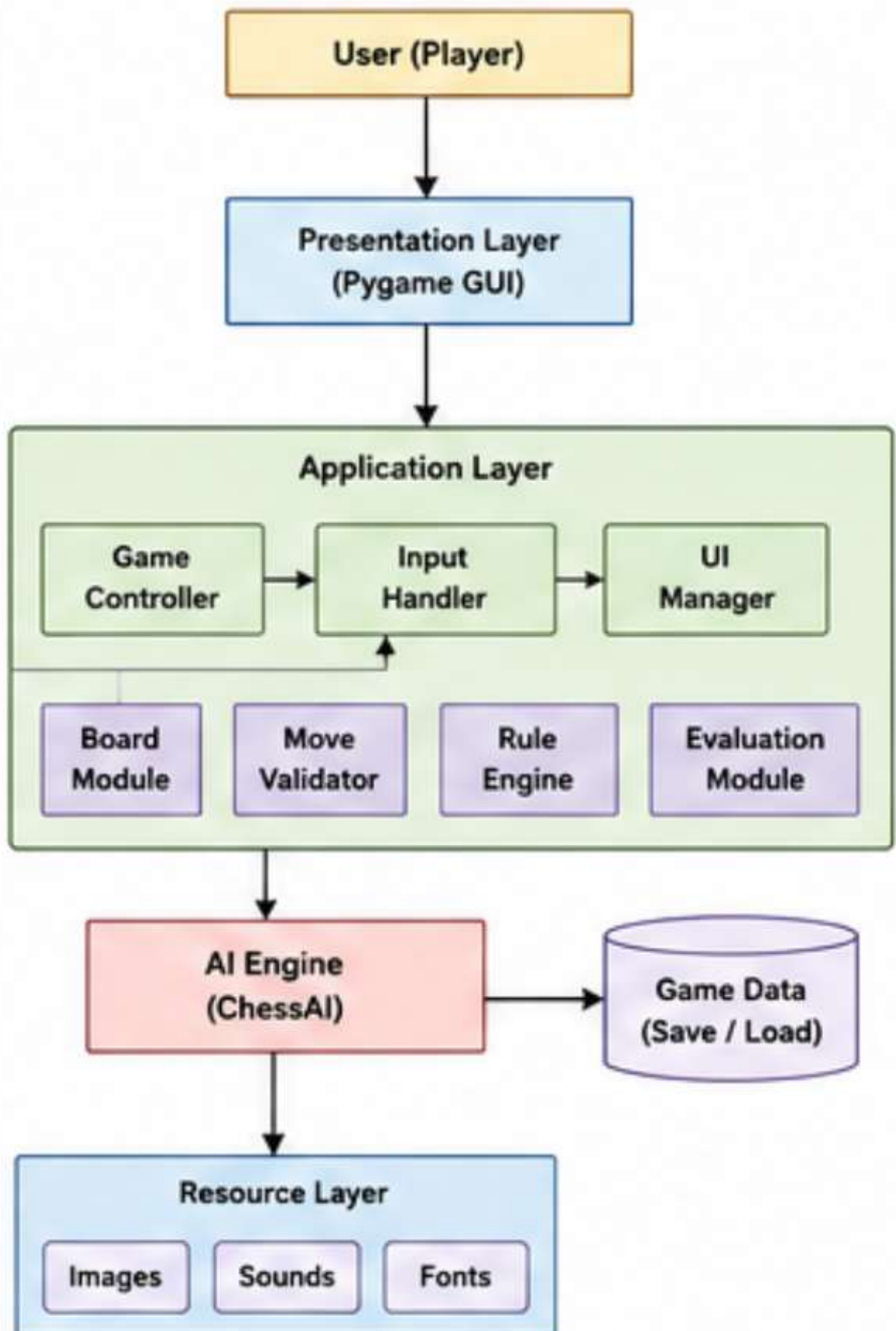
5.2 UML Class Diagram



5.3 Data Flow Diagram



5.4 System Architecture Diagram



5.5 Menu Design Wireframe

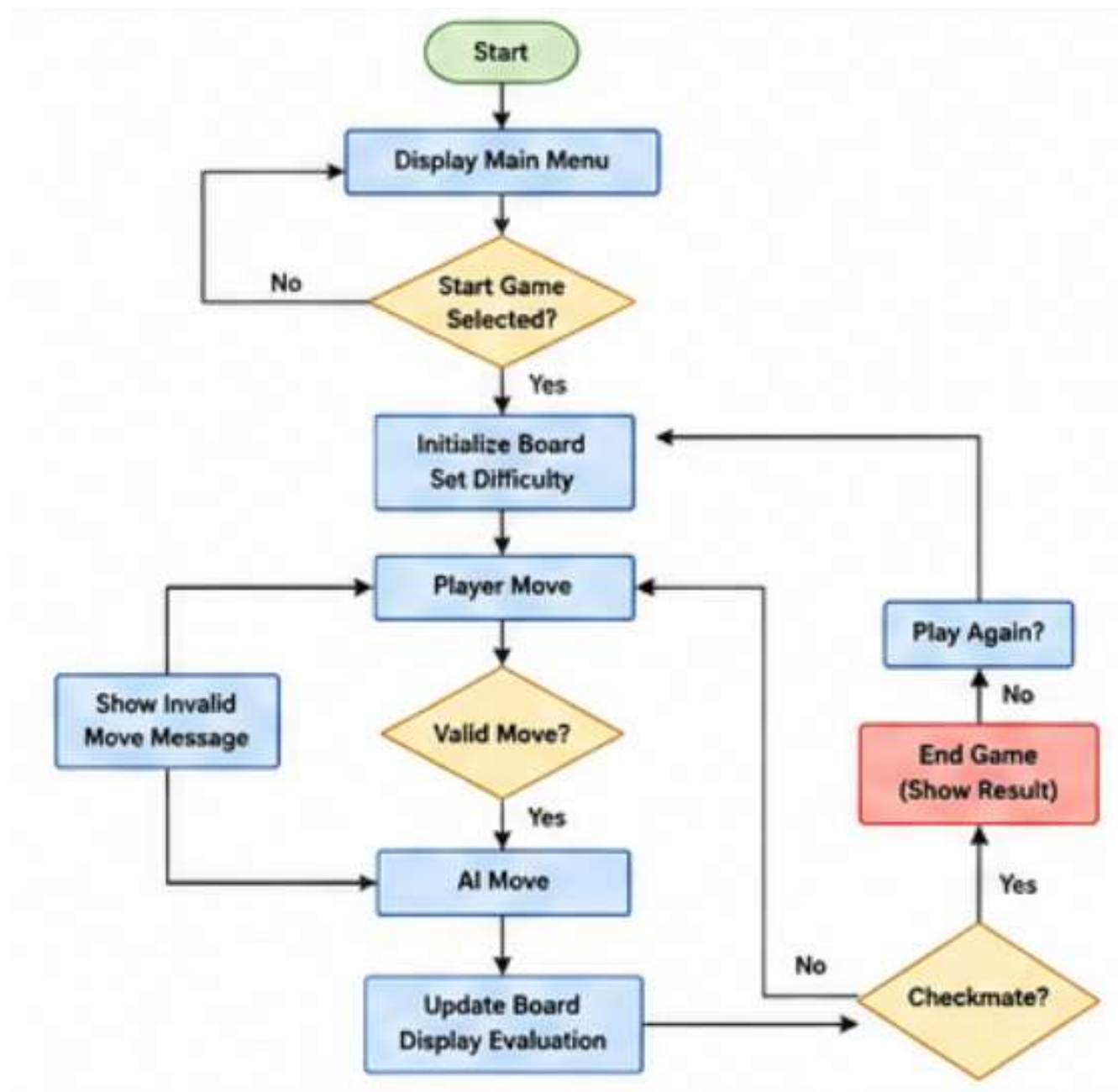


Implemented Pygame menu screen

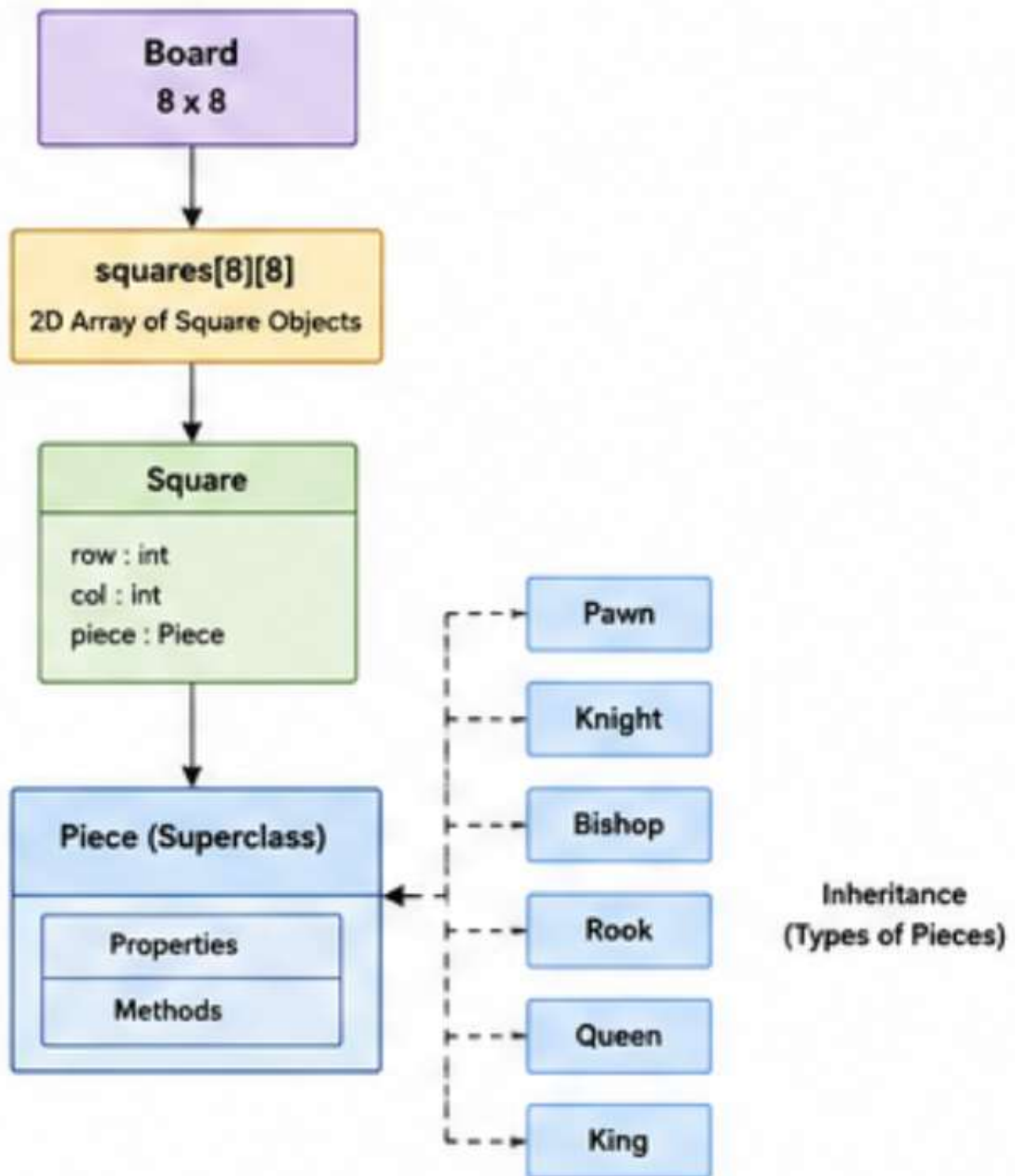
5.6 Interface Design (In-Game Screen)



5.7 Flowchart



5.8 Board Structure Diagram



6. Algorithms

Algorithms are a key part of the Chess AI Trainer because they control how the game processes moves, checks whether actions are valid, and allows the artificial intelligence opponent to respond. The main algorithms used in this project include move validation, AI move selection and board evaluation.

6.1 Move Validation Algorithm

The move validation algorithm ensures that the player can only make legal chess moves. When the user selects a piece, the system calculates all valid moves for that piece based on its type and position. For example, a bishop can only move diagonally, while a rook can only move horizontally or vertically.

Once the player selects a destination square, the program compares the attempted move with the list of valid moves. If the move exists in the list, the move is accepted and the board is updated. If the move is not valid, the system rejects the move and the player must choose again.

Pseudocode:

START

Player selects a piece

IF selected square contains a piece AND piece belongs to current player THEN

 Calculate legal moves for selected piece

 Store selected piece

 Store selected square

ENDIF

Player selects destination square

Create move using selected square and destination square

IF move is in legal moves list THEN

 Move piece to destination square

 Remove captured piece if destination contains enemy piece

 Update board state

```

    Change turn

ELSE

    Reject move

    Clear selected piece

ENDIF

END

```

6.2 AI Algorithm

The AI algorithm allows the computer to make a move after the player completes their turn. The AI first generates all possible legal moves for black pieces. It then evaluates each possible move using a simple scoring system.

In Easy mode, the AI may choose randomly from available legal moves. In Medium mode, the AI prioritises stronger moves, such as capturing valuable enemy pieces or developing useful pieces like knights and bishops. This creates a more realistic opponent while still keeping the algorithm simple enough to run efficiently on school hardware.

Pseudocode:

```

START
IF it is AI's turn THEN
    Get all legal moves for AI pieces
    IF difficulty is Easy THEN
        Choose random legal move
    ELSE IF difficulty is Medium THEN
        Set best score to very low number
        FOR each legal move
            Calculate move score
            IF move captures enemy piece THEN
                Add captured piece value to score
            ENDIF
            IF move develops knight or bishop THEN
                Add bonus score
            ENDIF
            IF move is better than current best move THEN
                Store move as best move
            ENDIF
        ENDFOR
        Choose one best move
    ENDIF
    Execute AI move
    Update board
    Change turn
ENDIF
END

```

6.3 Evaluation Algorithm

The evaluation algorithm calculates which player has an advantage by assigning values to pieces on the board. Each piece is given a numerical value based on its importance in chess.

Piece Value

Pawn	1
Knight	3
Bishop	3
Rook	5
Queen	9
King	10000

The algorithm loops through every square on the board. If a square contains a piece, the value of that piece is added or subtracted depending on which side it belongs to. The final score is displayed as the evaluation score.

A positive score indicates an advantage for one side, while a negative score indicates an advantage for the opposing side. This helps users understand the current state of the game.

Pseudocode:

START

Set score to 0

FOR each row on board

FOR each column on board

IF square contains a piece THEN

Get value of piece

IF piece belongs to AI THEN

Add value to score

ELSE

Subtract value from score

ENDIF

ENDIF

ENDFOR

ENDFOR

Display evaluation score

END

6.4 Importance of Algorithms

These algorithms allow the Chess AI Trainer to function as more than a simple graphical board. Move validation ensures the game follows chess rules, the AI algorithm allows single-player gameplay, and the evaluation algorithm provides feedback to the user. Together, these algorithms demonstrate problem-solving, logic design and artificial intelligence concepts within the project.

7. Development Log

Month 1 – Project Selection and Research

- Investigated possible project ideas for the Software Engineering major project.
- Chose to develop a Chess AI Trainer using Python and Pygame.
- Researched chess game development tutorials and existing chess software.
- Analysed project requirements and feasibility.
- Created initial project proposal and scope.
- Began researching AI algorithms suitable for chess.

Achievements

- Project idea approved.
- Software tools selected (Python, Pygame, VS Code, GitHub).

Challenges

- Limited experience with game development.
- Needed to learn Pygame framework.

Month 2 – Planning and Design

- Created project timeline and Gantt chart.
- Developed use case diagram.
- Created UML class diagram.
- Designed system architecture.
- Planned board representation and piece movement logic.
- Designed menu system and user interface.

Achievements

- System design completed.
- Project structure finalised.

Challenges

- Determining how classes would interact.
- Planning future AI integration.

Month 3 – Board and Piece Development

- Created Board class.
- Implemented Square class.
- Created Piece superclass.
- Created Pawn, Knight, Bishop, Rook, Queen and King subclasses.
- Added chess board display.
- Added chess piece textures.

Achievements

- Chess board successfully rendered.
- Pieces displayed correctly.

Challenges

- Positioning pieces accurately.
- Managing image assets.

Month 4 – Move Validation System

- Implemented legal move generation.
- Added pawn movement rules.
- Added rook movement rules.
- Added bishop movement rules.
- Added queen movement rules.
- Added knight movement rules.
- Added king movement rules.
- Implemented piece selection system.

Achievements

- Players could move pieces legally.

Challenges

- Preventing illegal moves.
- Debugging move calculations.

Month 5 – Advanced Chess Features

- Implemented castling.
- Implemented pawn promotion.
- Implemented en passant.
- Added move history tracking.
- Added move highlighting.
- Added board evaluation display.

Achievements

- Most official chess rules implemented.

Challenges

- En passant logic caused bugs.
- Castling required multiple conditions.

Month 6 – Artificial Intelligence Development

- Created ChessAI class.
- Developed board evaluation system.
- Assigned piece values:
 - Pawn = 1
 - Knight = 3
 - Bishop = 3
 - Rook = 5
 - Queen = 9
 - King = 10000
- Implemented AI move generation.
- Added difficulty selection system.

Achievements

- AI successfully made moves.

Challenges

- AI repeatedly moved pieces back and forth.
- Difficulty balancing.

Month 7 – User Interface Improvements

- Developed game menu.
- Added difficulty selection screen.
- Added sound effects.
- Added board themes.
- Added evaluation display.
- Improved game appearance.

Achievements

- Professional-looking interface created.

Challenges

- Screen sizing issues.
- Layout adjustments.

Month 8 – Testing and Debugging

- Conducted functional testing.
- Tested all chess pieces.
- Tested AI responses.
- Fixed move validation bugs.
- Fixed pawn capture issues.
- Fixed AI movement issues.
- Fixed evaluation display errors.
- Tested different difficulty levels.

Achievements

- Major bugs resolved.

Challenges

- Tap-to-move implementation.
- Checkmate detection errors.

Month 9 – Finalisation and Documentation

- Added game-over screen.
- Improved AI behaviour.
- Completed project documentation.
- Created diagrams and supporting evidence.
- Prepared presentation slides.
- Completed evaluation and reflection.

Achievements

- Fully playable chess game completed.
- Documentation finalised.

Challenges

- Ensuring stability before submission.
- Verifying all requirements were met.

Reflection on Development

Throughout the project, I developed skills in object-oriented programming, game development, debugging and artificial intelligence. The most challenging aspect was implementing legal chess rules while maintaining game stability. Through extensive testing and problem solving, I successfully created a functional Chess AI Trainer that met the project objectives. The project also improved my ability to plan long-term software development and manage a large-scale coding project over nine months.

8. Testing

Testing was conducted throughout development to ensure the Chess AI Trainer functioned correctly. Multiple testing methods were used, including black box testing, white box testing, boundary testing and user testing.

8.1 Test Table

Test Case	Expected Result	Actual Result	Status
Pawn move forward 1 square	Pawn moves forward	Pawn moved correctly	✓ Pass
Pawn move forward 2 squares	Pawn moves 2 squares on first move	Worked correctly	✓ Pass
Pawn capture	Pawn captures diagonally	Worked correctly	✓ Pass
Knight movement	Knight moves in L shape	Worked correctly	✓ Pass
Bishop movement	Bishop moves diagonally	Worked correctly	✓ Pass
Rook movement	Rook moves horizontally/vertically	Worked correctly	✓ Pass
Queen movement	Queen combines rook and bishop moves	Worked correctly	✓ Pass
King movement	King moves one square	Worked correctly	✓ Pass
Castling	King and rook move correctly	Worked correctly	✓ Pass
Pawn promotion	Pawn becomes queen	Worked correctly	✓ Pass

Test Case	Expected Result	Actual Result	Status
En passant	Special capture performed	Worked correctly	✓ Pass
Piece capture	Captured piece removed	Worked correctly	✓ Pass
AI response	AI makes move after player	Worked correctly	✓ Pass
Difficulty selection	Difficulty changes AI behaviour	Worked correctly	✓ Pass
Evaluation display	Score displayed correctly	Worked correctly	✓ Pass
Menu screen	Menu loads successfully	Worked correctly	✓ Pass
Board reset	New game created	Worked correctly	✓ Pass
Theme change	Board colours change	Worked correctly	✓ Pass
Invalid move	Move rejected	Worked correctly	✓ Pass
Game exit	Window closes safely	Worked correctly	✓ Pass

8.2 Black Box Testing

Black box testing focuses on testing the functionality of the software without examining the internal code.

Example 1 – Pawn Movement

Input: Move pawn from e2 to e4

Expected Result: Pawn moves forward two squares

Actual Result: Pawn moved correctly

Outcome: Pass

Example 2 – Invalid Move

Input: Attempt to move bishop through another piece

Expected Result: Move rejected

Actual Result: Move rejected

Outcome: Pass

Example 3 – Reset Game

Input: Press R key

Expected Result: Board returns to starting position

Actual Result: Board reset successfully

Outcome: Pass

Benefits

- Tests user functionality.
- Simulates real user interactions.
- Easy to perform.

8.3 White Box Testing

White box testing involves examining the internal code structure and logic.

Example 1 – AI Evaluation Function

Code Tested:

```
score += piece.value
```

Purpose: Verify piece values are added correctly.

Result: Evaluation score updated correctly.

Example 2 – Turn Switching

Code Tested:

```
self.next_player = 'white' if self.next_player == 'black' else 'black'
```

Purpose: Verify turns alternate correctly.

Result: Player turns changed correctly.

Example 3 – Move Validation

Code Tested:

```
return move in piece.moves
```

Purpose: Ensure only legal moves are accepted.

Result: Illegal moves rejected successfully.

8.4 Boundary Testing

Boundary testing checks how the program behaves at the limits of valid input.

Board Edge Testing

Test: Move rook to edge of board.

Expected Result: Rook stops at board boundary.

Actual Result: Correct.

Outcome: Pass.

Corner Square Testing

Test: Move piece into corner square.

Expected Result: Piece remains within board.

Actual Result: Correct.

Outcome: Pass.

Invalid Coordinates

Test: Click outside board area.

Expected Result: No move occurs.

Actual Result: No move occurred.

Outcome: Pass.

First and Last Rows

Test: Promote pawn on final rank.

Expected Result: Pawn becomes queen.

Actual Result: Correct.

Outcome: Pass.

8.5 User Testing

User testing was conducted by allowing students and friends to play the game and provide feedback.

User 1

Feedback:

- Game was easy to understand.
- AI responded quickly.

Suggested Improvement:

- Add more AI difficulties.

User 2

Feedback:

- Liked board evaluation feature.
- Menu was simple to use.

Suggested Improvement:

- Add timer feature.

User 3

Feedback:

- Piece movement felt smooth.
- Rules worked correctly.

Suggested Improvement:

- Add check and checkmate notifications.

8.6 Testing Summary

Testing confirmed that the Chess AI Trainer met its functional requirements. All major chess mechanics operated correctly, including movement validation, captures, castling, pawn promotion and AI responses. Testing also helped identify and resolve issues related to move validation, AI behaviour and game state management. The final version was stable and suitable for deployment and assessment.

9. Implementation

9.1 Overview

The implementation phase involved converting the system design into a fully functioning Chess AI Trainer using Python and Pygame. The project was developed using an object-oriented approach, where each major component of the game was represented by its own class. This improved maintainability, readability and future scalability.

The final solution consists of multiple source files including:

- main.py
- board.py
- ai.py
- game.py
- piece.py
- move.py
- square.py
- dragger.py
- config.py

Each file performs a specific task within the system.

9.2 main.py

The main.py file acts as the entry point of the application. It creates the game window, handles user interaction and controls the game loop.

Responsibilities include:

- Initialising Pygame
- Displaying the chess board
- Handling mouse clicks
- Selecting pieces
- Moving pieces
- Processing AI turns
- Updating the display

```
src > main.py > ...
1  import pygame
2  import sys
3
4  from const import *
5  from game import Game
6  from square import Square
7  from move import Move
8
9
10 class Main:
11
12     def __init__(self):
13         pygame.init()
14         self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
15         pygame.display.set_caption("ChessTrainer")
16         self.game = Game()
17
18     def make_ai_move(self, board, game):
19         if game.next_player == "black":
20             ai_result = game.ai.get_best_move(board)
```

Explanation

When the player clicks on a piece, the program calculates all legal moves. If the player selects a valid destination square, the move is executed and the AI automatically responds.

This file acts as the controller that connects all other components of the system.

9.3 board.py

The board.py file contains the core chess logic.

Responsibilities include:

- Creating the chess board
- Storing piece positions
- Calculating legal moves
- Detecting captures
- Castling
- En passant
- Pawn promotion
- Check detection
- Checkmate detection

```
src > board.py > Board > is_king_in_check
1  from const import *
2  from square import Square
3  from piece import *
4  from move import Move
5  from sound import Sound
6  import copy
7  import os
8
9  class Board:
10
11     def __init__(self):
12         self.squares = [[0, 0, 0, 0, 0, 0, 0, 0] for col in range(COLS)]
13         self.last_move = None
14         self._create()
15         self._add_pieces('white')
16         self._add_pieces('black')
17
18     def move(self, piece, move, testing=False):
19         initial = move.initial
20         final = move.final
```

Explanation

This is the most complex file in the project because it contains the rules of chess.

For example:

```
def valid_move(self, piece, move):
    return move in piece.moves
```

This method ensures that only legal moves are accepted by the game. The board class also manages special chess rules such as castling and pawn promotion.

9.4 ai.py

The ai.py file contains the computer opponent.

Responsibilities include:

- Evaluating board positions
- Generating possible moves
- Selecting the best move
- Supporting multiple difficulty levels

```
src > ai.py > ChessAI > get_best_move
1  import random
2
3  from const import *
4  from piece import Pawn, Knight, Bishop, Rook, Queen, King
5
6
7  class ChessAI:
8
9      def __init__(self, color="black", difficulty="medium"):
10         self.color = color
11         self.difficulty = difficulty
12
13     def evaluate_board(self, board):
14         score = 0
15
16         values = {
17             Pawn: 1,
18             Knight: 3,
19             Bishop: 3,
20             Rook: 5,
```

Explanation

The AI evaluates the board using material values:

Piece Value

Pawn	1
Knight	3
Bishop	3
Rook	5
Queen	9
King	10000

The AI searches through all legal moves and selects the move with the highest evaluation score.

The Medium difficulty uses evaluation-based selection while Easy difficulty can use more random move choices.

9.5 game.py

The game.py file controls visual elements and game states.

Responsibilities include:

- Displaying pieces
- Drawing the board
- Showing evaluation score
- Showing menus
- Handling themes
- Managing turns

```
src > game.py > Game
1  from ai import ChessAI
2  import pygame
3
4  from const import *
5  from board import Board
6  from dragger import Dragger
7  from config import Config
8  from square import Square
9
10 class Game:
11
12     def show_game_over(self, surface, winner):
13         font = pygame.font.SysFont("Arial", 60)
14
15         overlay = pygame.Surface((WIDTH, HEIGHT))
16         overlay.set_alpha(180)
17         overlay.fill((0, 0, 0))
18
19         surface.blit(overlay, (0, 0))
20
```

Explanation

This class acts as the bridge between the user interface and the game logic.

It controls:

- Menu screens
- Board themes
- Evaluation display
- Game over messages

Example:

```
self.ai = ChessAI("black", "medium")
```

This line creates the AI opponent when the game starts.

9.6 Key Features Implemented

AI Opponent

The AI automatically responds after every player move.

Benefits:

- Provides continuous gameplay
- Allows solo practice
- Demonstrates decision-making algorithms

Evaluation Bar

The evaluation system analyses the board and displays a score.

Example:

Eval: +3

This indicates White has approximately a three-point material advantage.

Benefits:

- Helps players understand positions
- Supports learning and improvement

Difficulty Selection

The menu allows players to select:

- Easy
- Medium

Benefits:

- Suitable for beginners
- Provides increasing challenge

Piece Movement System

Implemented:

- Legal move validation
- Capturing
- Castling
- En passant
- Pawn promotion

Benefits:

- Creates an authentic chess experience
- Ensures rules are enforced correctly

10. Evaluation

10.1 What went well

The project successfully achieved its objectives.

Key achievements include:

Functional Chess Game

All major chess mechanics were implemented successfully including:

- Piece movement
- Capturing
- Castling
- Promotion
- Turn management

Artificial Intelligence

The AI was able to:

- Analyse positions
- Evaluate material advantage
- Select competitive moves

This significantly improved the educational value of the project.

User Interface

The graphical interface developed using Pygame was simple and intuitive.

Features included:

- Chess board display
- Piece graphics
- Evaluation display
- Difficulty menu

Users were able to learn the controls quickly.

Object-Oriented Design

The use of classes improved:

- Maintainability
- Readability
- Scalability

Future improvements can be added without major redesign.

10.2 Problems Encountered

Several technical challenges were encountered during development.

Checkmate Bugs

One of the most difficult problems involved checkmate detection.

Issues included:

- False checkmate detection
- Checkmate triggering too early
- Missing stalemate situations

These problems required extensive debugging of move validation and king safety calculations.

King Movement Bugs

At various stages the king was unable to move legally.

Causes included:

- Incorrect check detection
- Castling conflicts
- Move generation errors

This required significant testing and code refinement.

AI Selection Bugs

The AI occasionally:

- Repeated moves
- Selected poor moves
- Failed to recognise advantageous captures

These issues highlighted limitations in the simple evaluation system.

Tap-to-Move System

Implementing tap-to-move controls introduced several issues including:

- Piece selection errors
- Move validation conflicts
- Interaction bugs with dragging functionality

Eventually the system was redesigned to improve usability.

10.3 Future Improvements

Although the project is fully functional, there are many opportunities for improvement.

Minimax Algorithm

The current AI evaluates positions only one move ahead.

Future versions could implement:

- Minimax Search

Benefits:

- Stronger AI
- Better strategic decisions

Alpha-Beta Pruning

Alpha-beta pruning would improve Minimax efficiency.

Benefits:

- Faster calculations
- Deeper search depth
- Stronger gameplay

Move History

A move history panel could record every move.

Benefits:

- Game analysis
- Learning support
- Replay functionality

Undo Button

Players could reverse mistakes.

Benefits:

- Improved usability
- Better learning experience

Online Multiplayer

Players could compete against others over the internet.

Benefits:

- Increased engagement
- Competitive gameplay
- Greater replay value

10.4 Final Evaluation

Overall, the Chess AI Trainer successfully met the original project objectives. The system provides a playable chess experience with automated AI responses, evaluation scoring and multiple difficulty levels. Despite challenges involving checkmate detection, king movement and AI decision making, these issues were largely resolved through testing and iterative development. The project demonstrates strong programming, problem-solving and software engineering skills while providing a solid foundation for future enhancements.

11. References

Python Software Foundation. (2025). *Python Documentation*.

Retrieved from <https://docs.python.org/3/>

Flask Documentation. (2025). *Flask Web Framework*. Retrieved from

<https://flask.palletsprojects.com/>

SQLite Documentation. (2025). *SQLite Documentation*. Retrieved

from <https://www.sqlite.org/docs.html>

GitHub. (2025). *GitHub Documentation*. Retrieved from

<https://docs.github.com/>

Visual Studio Code. (2025). *VS Code Documentation*. Retrieved from

<https://code.visualstudio.com/docs>

bcrypt Documentation. (2025). *bcrypt Library*. Retrieved from

<https://pypi.org/project/bcrypt/>

Chess Programming Wiki. (2025). *Chess Programming Wiki*. Retrieved

from <https://www.chessprogramming.org/>

Google Lighthouse. (2025). *Lighthouse Documentation*. Retrieved

from <https://developer.chrome.com/docs/lighthouse/>

YouTube Tutorial:

Build a Chess Engine in Python. (Year). Retrieved from

<https://www.youtube.com/watch?v=OpL0Gcfn4B4>

AI and Third-Party Code Usage

This project utilised an online tutorial to assist in the development of the chess engine component. The original implementation was modified and integrated into a web-based application with additional features, including user authentication, database storage, and interface design.

All external code sources were reviewed, adapted, and extended to meet the requirements of the project.

12. Appendix

The Appendix contains supporting evidence that demonstrates the planning, design, development and testing of the Chess AI Trainer project. These materials provide proof of the work completed throughout the Software Development Life Cycle.

Appendix A – Source Code Excerpts

The following code excerpts show important sections of the program.

A.1 AI Evaluation Function

File: **ai.py**

```
piece_values = {  
    Pawn: 1,  
    Knight: 3,  
    Bishop: 3,  
    Rook: 5,  
    Queen: 9,  
    King: 10000}
```

Purpose:

Assigns a numerical value to each chess piece, allowing the AI to evaluate board positions and determine which moves are advantageous.

A.2 Move Validation

File: **board.py**

```
def valid_move(self, piece, move):  
    return move in piece.moves
```

Purpose:

Ensures only legal chess moves are accepted by the game.

A.3 Turn Management

File: **game.py**

```
def next_turn(self):  
    self.next_player = 'white' if self.next_player == 'black' else 'black'
```

Purpose:

Alternates turns between the player and the AI.

A.4 AI Move Selection

File: **ai.py**

```
ai_result = game.ai.get_best_move(board)
```

Purpose: Generates the AI's move after the player completes a turn.

Appendix B – Screenshots

The following screenshots should be included as evidence of the completed project.

Screenshot 1 – Main Menu



Description:

Displays difficulty selection options before gameplay begins.

Caption:

Figure B1: Difficulty Selection Menu.

Screenshot 2 - Gameplay Screen



Description:

Shows the chess board, pieces and evaluation display during gameplay.

Caption:

Figure B2: Main Gameplay Interface.

Screenshot 3 – AI Move Example



Description:

Demonstrates the AI responding to a player move.

Caption:

Figure B3: AI Automatically Responding.

Screenshot 4 – Piece Capture



Description:

Shows a piece being captured and removed from the board.

Caption:

Figure B4: Piece Capture Functionality.

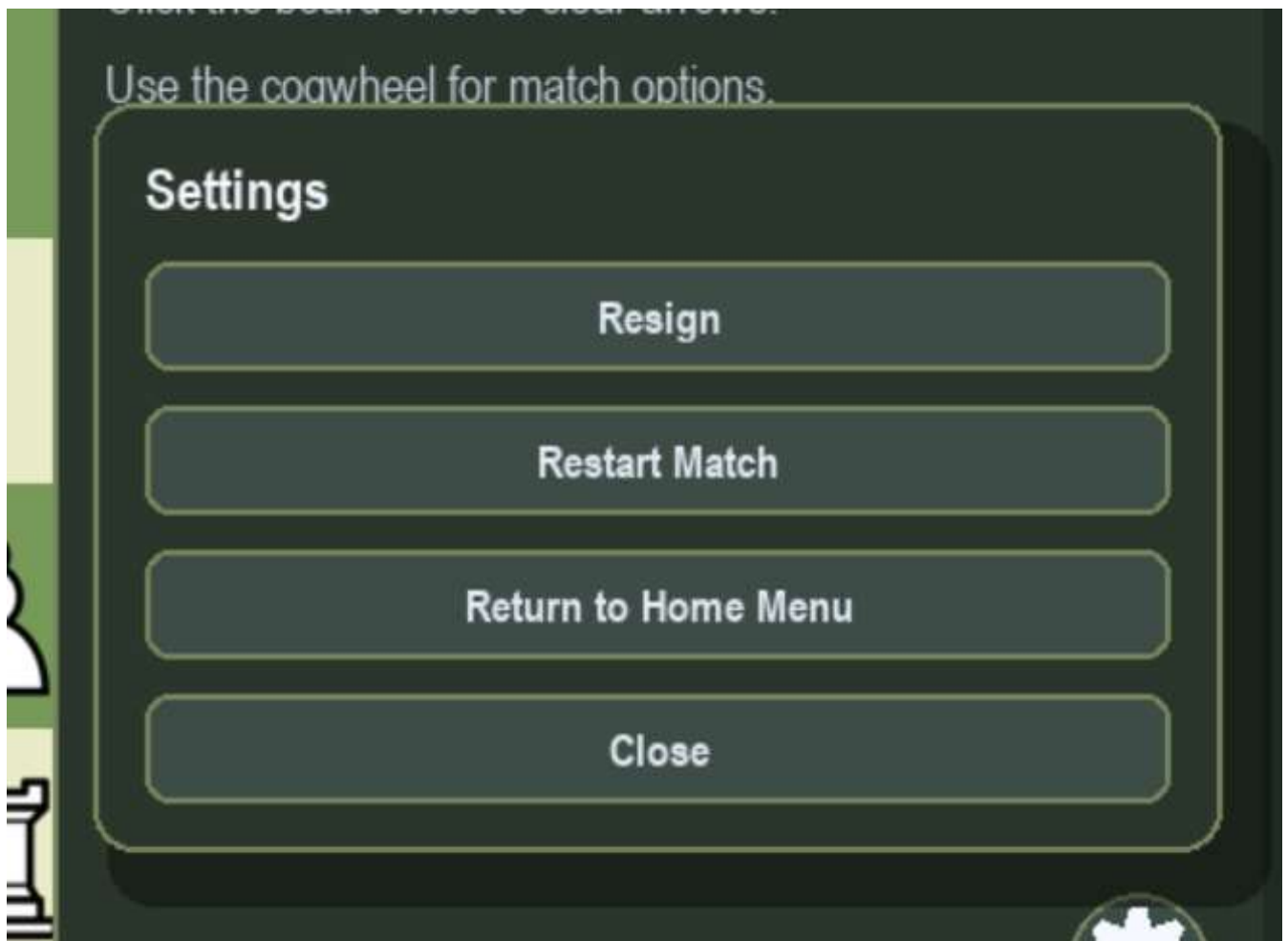
Screenshot 5 – Checkmate Detection



Description:
Shows the checkmate detection.

Caption:
Figure B5: Checkmate Detection.

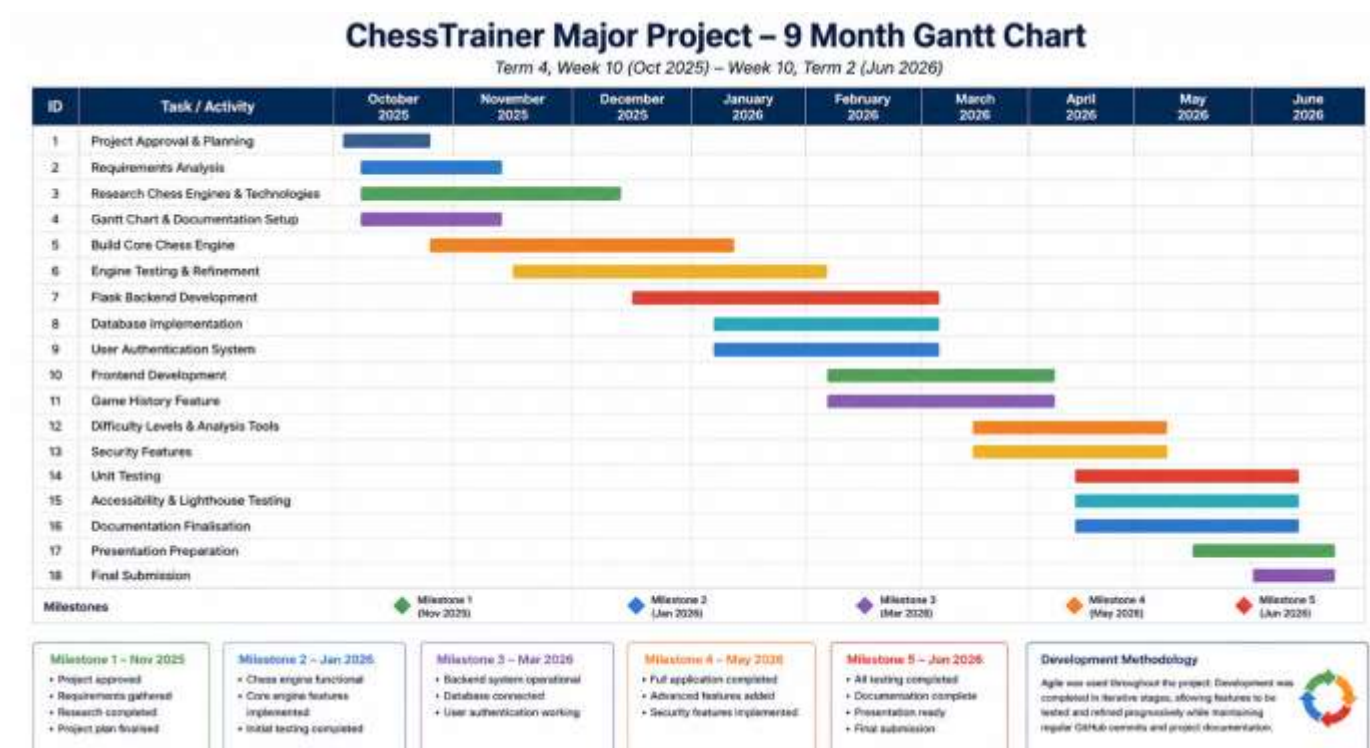
Screenshot 6 – Game Reset



Description:
Shows the board after pressing the reset key.

Caption:
Figure B6: Board Reset Function.

Appendix C – Gantt Chart



The Gantt chart outlines all major development stages completed over the nine-month development period.

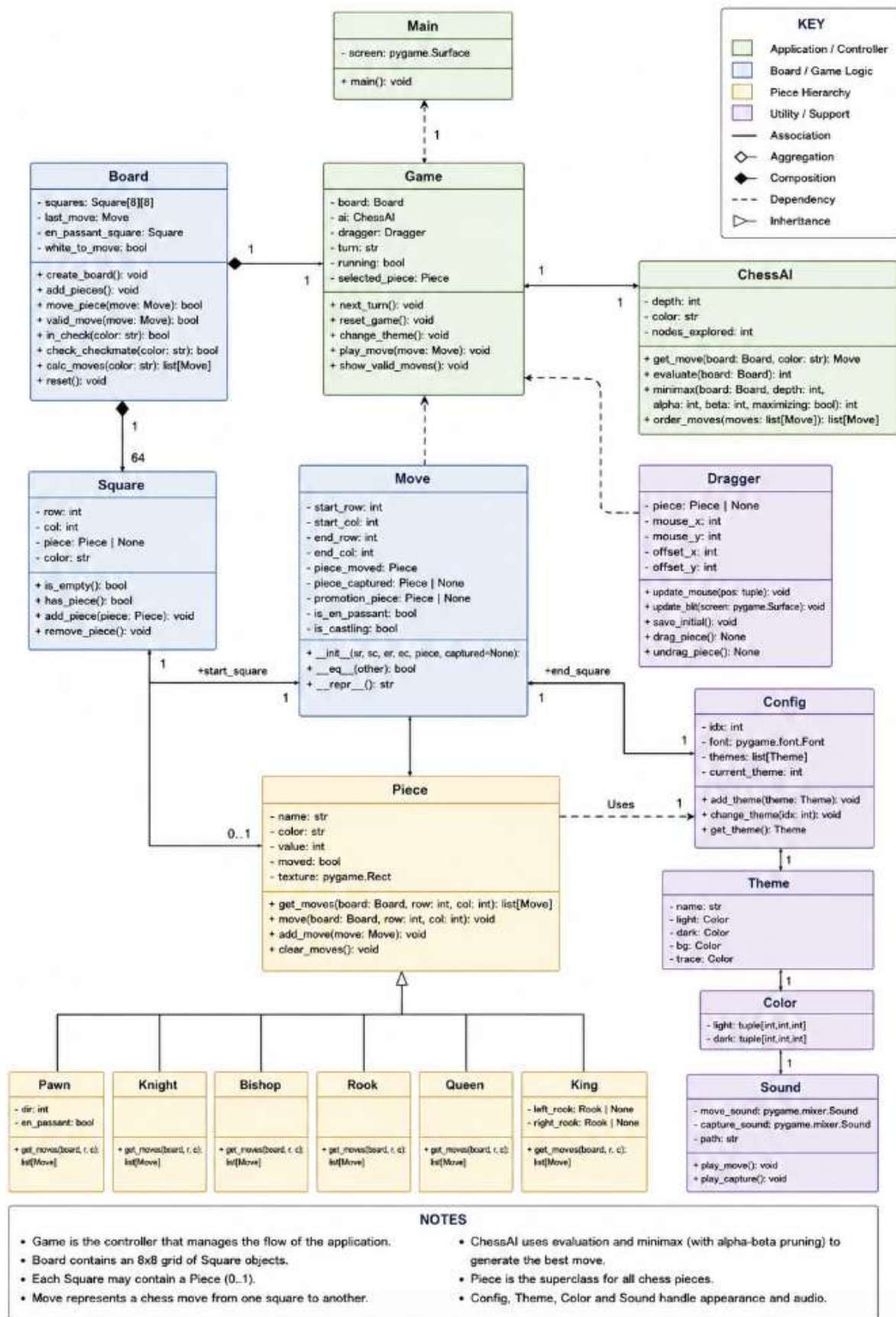
Activities Included

- Requirements Analysis
- Research
- System Design
- Development
- AI Implementation
- Testing
- Documentation
- Evaluation
- Final Submission

Caption:

Figure C1: Project Gantt Chart.

Appendix D – UML Class Diagram



Classes included:

- Main
- Game
- Board
- Piece
- Pawn
- Knight
- Bishop
- Rook
- Queen
- King
- Move
- Dragger
- ChessAI

Relationships shown:

- Inheritance
- Composition
- Associations

Caption:

Figure D1: UML Class Diagram.

Appendix Summary

The appendix provides supporting evidence for the development of the Chess AI Trainer. It includes source code excerpts, screenshots, planning documentation and UML diagrams. These materials demonstrate that the project was planned, developed and tested according to Software Engineering best practices and support the conclusions made throughout the documentation.

Code Documentation

- The source code follows a 3:1 comment-to-code ratio where practical, ensuring each major block of code is explained.
- Comments describe the purpose of classes, functions and complex algorithms rather than repeating obvious code.
- Inline comments are used to explain AI decision-making, event handling and game logic.
- Meaningful variable and function names reduce the need for excessive comments.
- Consistent documentation makes the project easier to maintain, debug and extend by other developers.

Throughout development, the source code was documented using a consistent commenting standard. Approximately one comment was included for every three lines of code where appropriate. Comments explain the purpose of functions, algorithms and complex sections of logic, improving readability, maintainability and future modifications. Clear documentation also supports client handover by making the program easier for another developer to understand.